math

1. 基础数学(高中部分) ---------- 2
  1.1. 基础定义 ---------- 2
    1.1.1.1. 三角函数 ---------- 2
  1.2. 三角恒等变换 ---------- 3
    1.2.1. 基本公式 ---------- 3
    1.2.2. 负角 ---------- 4
2. 数论 ---------- 4
  2.1. 数论基础 ---------- 4
  2.2. 素数 ---------- 4
    2.2.1. 素性检测 ---------- 4
      2.2.1.1. Miller-Rabin 素性测试 ---------- 4
      2.2.1.2. 二次探测定理 ---------- 5
  2.3. 最大公约数 ---------- 6
    2.3.1. 更相减损术 ---------- 6
    2.3.2. 欧几里得算法 ---------- 6
      2.3.2.1. 正确性证明 ---------- 7
      2.3.2.2. 时间复杂度 ---------- 7
      2.3.2.3. 复杂度证明 ---------- 7
    2.3.3. 裴蜀定理 ---------- 7
    2.3.4. 拓展欧几里得算法 ---------- 8
      2.3.4.1. 求 1-n 的因数 ---------- 8
    2.3.5. 大整数的 gcd ---------- 8
  2.4. 乘法逆元 ---------- 9
    2.4.1. 定义 ---------- 9
    2.4.2. 快速幂求逆元 ---------- 9
    2.4.3. exgcd 求逆元 ---------- 9
    2.4.4. 线性求1~n的逆元 ---------- 10
    2.4.5. 线性求任意 n 个数的逆元 ---------- 10
  2.5. 欧拉函数 ---------- 11
    2.5.1. 定义 ---------- 11
    2.5.2. 性质 ---------- 11
    2.5.3. 实现 ---------- 12
  2.6. 费马小定理 && 欧拉定理 ---------- 13
    2.6.1. 费马小定理 ---------- 13
    2.6.2. 证明 ---------- 13
    2.6.3. 欧拉定理 ---------- 14
    2.6.4. 拓展欧拉定理(欧拉降幂) ---------- 14
  2.7. 线性同余方程 ---------- 15
  2.8. 线性同余方程组 ---------- 16

2.9. 中国剩余定理(crt) ---------- 16
2.9.1. 定理 ---------- 16
2.9.2. 过程 ---------- 17
2.10. 卢卡斯定理 ---------- 17
2.11. 分解质因数 ---------- 18
2.11.1. Pollard Rho 算法 ---------- 18
2.12. 筛法 ---------- 21
2.12.1. 线性筛 ---------- 21
3. 组合数学 ---------- 21
3.1. 排列组合 ---------- 21
3.1.1. 求组合数 ---------- 21
3.1.2. 插板法 ---------- 23
3.1.3. 组合数性质/二项式推论 ---------- 23
3.2. 容斥原理 ---------- 24
3.3. 斯特林数 ---------- 24
3.4. 范德蒙卷积 ---------- 24
3.5. 卡特兰数 ---------- 24
3.6. 分拆数 ---------- 25
3.6.1. 定义 ---------- 25
3.6.2. k 部分分拆数 ---------- 25
3.6.3. 互异分拆数 ---------- 25
4. 博弈论 ---------- 26
4.1. 基础知识 ---------- 26
4.2. 公平组合游戏 ---------- 26
4.2.1. Nim 游戏 ---------- 26
4.2.2. SG 函数 ---------- 26
5. 线性代数 ---------- 27
5.1. 矩阵 ---------- 27
6. 高等数学 ---------- 28
6.1. 调和级数 ---------- 28
7. 多项式与生成函数 ---------- 29
7.1. 积性函数 ---------- 29
8. 结论 ---------- 29

1. 基础数学(高中部分)

1.1. 基础定义

1.1.1.1. 三角函数

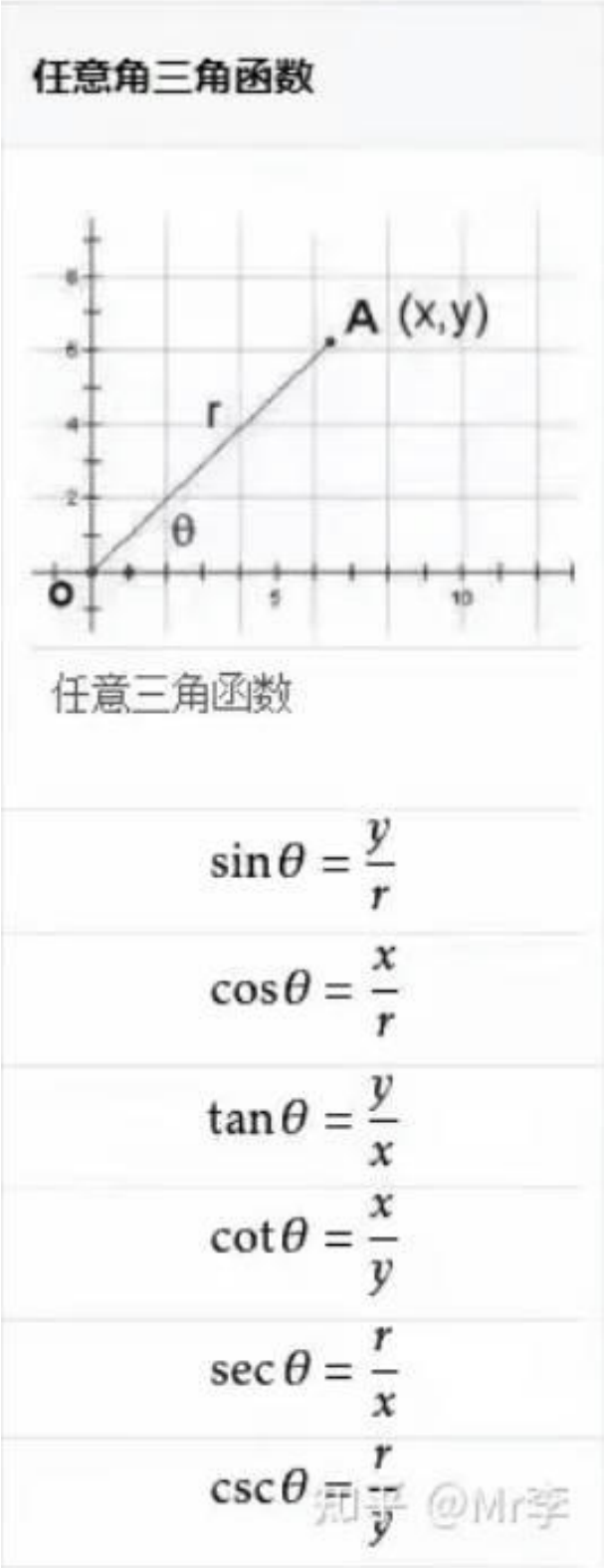

Figure 1: 三角函数定义

1.2. 三角恒等变换

https://www.zhihu.com/question/401463947

1.2.1. 基本公式

$$\cos^2\theta + \sin^2\theta = 1$$

$$1 + \tan^2\theta = \sec^2\theta$$

$$1 + \cot^2\theta = \csc^2\theta$$

1.2.2. 负角

$$\sin(-x) = -\sin x$$

$$\cos(-x) = \cos(x)$$

$$\tan(-x) = -\tan x$$

$$\sec(-x) = \sec(x)$$

$$\csc(-x) = -\csc(x)$$

$$\cot(-x) = -\cot(x)$$

2. 数论

2.1. 数论基础

$n \leq$	10^1	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\max\{\omega(n)\}$	2	3	4	5	6	7	8	8	9
$\max\{d(n)\}$	4	12	32	64	128	240	448	768	1344
$n \leq$	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$\max\{\omega(n)\}$	10	10	11	12	12	13	13	14	15
$\max\{d(n)\}$	2304	4032	6720	10752	17280	26880	41472	64512	103680

Figure 2: 因子种类数与因子数表

质数一定满足:6n+1,6n-1

2.2. 素数

2.2.1. 素性检测

判断一个数是不是素数

2.2.1.1. Miller-Rabin 素性测试

前置知识:费马小定理

对于一个质数n,任意正整数都满足$a^{n-1} \equiv 1 (\bmod n)$

Miller-Rabin 是基于其逆定理,来计算的

2.2.1.2. 二次探测定理

$x^2 \equiv 1 (\bmod p)$

$(x-1)(x+1) \equiv 0 (\bmod p)$

$x = 1$或-1

$(x-1)(x+1) = k * p$

因为p是质数,所以$p = x - 1$或$p = x + 1$

x 在模p意义下也就是1或$n - 1$

```cpp
ll qmi(__int128_t a, ll b, ll p)
{
    ll res = 1;
    while (b)
    {
        if (b & 1)
            res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}
bool MR(ll n)
{
    if (n == 2)
        return true;
    if (n <= 1 || n % 2 == 0)
        return false;
    const ll base[7] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    ll u = n - 1, k = 0;
    while (u % 2 == 0)
        k++, u /= 2;
    for (auto &x : base)
    {
        if (x % n == 0)
            continue;
```

```cpp
        ll v = qmi(x, u, n);
        if (v == 1 || v == n - 1)
            continue;
        for (int j = 1; j <= k; j++)
        {
            ll lst = v;
            v = (__int128_t)v * v % n;
            if (v == 1)
            {
                if (lst != n - 1)
                    return false;
                break;
            }
        }
        if (v != 1)
            return false;
    }
    return true;
}
```

2.3. 最大公约数

1.结合律:

$$\gcd(a, b, c) = \gcd(\gcd(a, b), c)$$

2.

$$a|bc, \gcd(a, b) = 1 \Rightarrow a|c$$

2.3.1. 更相减损术

时间复杂度

$O(\max(a, b))$

```cpp
ll gcd(ll a, ll b)
{
    if (a == b)
        return a;
    return (a > b ? gcd(a - b, b) : gcd(a, b - a));
}
```

2.3.2. 欧几里得算法

```cpp
ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
```

2.3.2.1. 正确性证明

设$a = k * b + c$

充分:

设$d|a, d|b$

那么有:

$\frac{a}{d} = \frac{b}{d} * k + \frac{c}{d}$

$\frac{c}{d} = \frac{a}{d} - \frac{b}{d} * k$

左边都能整除,所以$\frac{c}{d}$也是整数,即$d|c$

所以a, b的 gcd 也是$a\%b$的因数

必要:

设$d|b, d|c$

那么有:

$\frac{a\%b}{d} = \frac{c}{d} = \frac{a-b*k}{d} = \frac{a}{d} - \frac{b}{d} * k$

$\frac{a}{d} = \frac{c}{d} + \frac{b}{d} * k$

因为$d|b, d|c$,所以$d|a$

2.3.2.2. 时间复杂度

$O(\log(\max(a, b)))$

2.3.2.3. 复杂度证明

设$a > b$

$\gcd(a, b) = \gcd(b, a\%b)$

1.如果$b \geq \frac{a}{2}$

$a\%b = a - b \leq \frac{a}{2}$

2.如果$b < \frac{a}{2}$

$a\%b < \frac{a}{2}$

所以每次递归,其中一个数至少减小一半,复杂度为$2\log n$,忽略常数得到

$O(\log(\min(a, b)))$

2.3.3. 裴蜀定理

$$\gcd(a, b)|ax + by$$

$$\exists x, y \leftrightarrow ax + by = \gcd(a, b)$$

2.3.4. 拓展欧几里得算法

参考资料:https://zhuanlan.zhihu.com/p/647843184

如果$b \neq 0$,那么求出的可行解一定满足$|x| < b, |y| < a$

设 exgcd 得到的一组特解为x_0, y_0,那么存在通解为:

$x' = x_0 + \frac{b}{\gcd(a,b)}t$

$y' = y_0 - \frac{a}{\gcd(a,b)}t$

$t = ..., -2, -1, 0, 1, 2, ..$

```cpp
ll exgcd(ll a, ll b, ll &x, ll &y)
{
    if (!b)
    {
        x = 1, y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}
```

2.3.4.1. 求 1-n 的因数

```cpp
vector<vector<int>> fac(n + 1);
for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j += i)
        fac[j].push_back(i);
```

2.3.5. 大整数的 gcd

时间复杂度:$O\left(\log(\max\{a, b\})^2\right)$

1.$a = 0 \vee b = 0, (0, b) = b, (a, 0) = a$

2.$a\%2 == 0 \wedge b\%2 == 0, (a, b) = \left(\frac{a}{2}, \frac{b}{2}\right)$

3.$a\%2 == 0 \wedge b\%2 == 1, (a, b) = \left(\frac{a}{2}, b\right)$

4.$a\%2 == 1 \wedge b\%2 == 0, (a, b) = \left(a, \frac{b}{2}\right)$

4.$a\%2 == 1 \wedge b\%2 == 1, (a, b) = (b, a - b)$

2.4. 乘法逆元

2.4.1. 定义

如果线性同余方程$ax \equiv 1$,则称x为$a \bmod b$的逆元,记作a^{-1}

2.4.2. 快速幂求逆元

快速幂法使用了费马小定理，要求 mod 是一个素数

```cpp
ll qmi(ll a, ll b = mod - 2)
{
    ll res = 1;
    a %= mod;
    while (b)
    {
        if (b & 1)
            res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

优化常数版

```cpp
int qmi(int a, ll b, int p) {
    int res = 1;
    for (; b; b /= 2, a = 1ll * a * a % p) {
        if (b % 2) {
            res = 1LL * res * a % p;
        }
    }
    return res;
}
```

2.4.3. exgcd 求逆元

a的逆元a^{-1}一定满足$ax \equiv 1 \pmod{p}$

因为$ax \bmod p = ax - \frac{ax}{p} * p$

所以$ax \equiv 1 \pmod{p}$等价于

$$ax + p\left(-\frac{ax}{p}\right) \equiv 1 \pmod{p}$$

令$y=-\frac{ax}{p}$得

$$ax+py\equiv 1 \pmod{p}$$

用 exgcd 求解即可

注意只有当 gcd(a,mod)=1 时成立

2.4.4. 线性求$1\sim n$的逆元

显然

$$1\times 1\equiv 1(\mathrm{mod}\ \ p)$$

所以 1 的逆元是 1

对于求 i 的逆元 i^{-1}

令$k=\left\lfloor\frac{p}{i}\right\rfloor, j=p \bmod i$

则有

$$p=ki+j$$

在 mod p 意义下就有

$$ki+j\equiv 0 \pmod{p}$$

两边同时乘以$i^{-1}j^{-1}$

$$kj^{-1}+i^{-1}\equiv 0 \pmod{p}$$

移项

$$i^{-1}\equiv -kj^{-1} \pmod{p}$$

代入

$$i^{-1}\equiv -\left\lfloor\frac{p}{i}\right\rfloor(p \bmod i)^{-1} \pmod{p}$$

因为递推,显然$(p \bmod i)$已知

```cpp
inv[1] = 1;
for (int i = 2; i <= n; i++)
    inv[i] = (mod - mod / i) * inv[mod % i] % mod;
```

2.4.5. 线性求任意 **n** 个数的逆元

先求n个数的前缀积s_i,然后求出s_n的逆元(快速幂,exgcd 求),记作sv_n

然后从后往前,依次让$sv_i=sv_{i+1}*a_i$

这样a_i的逆元a_i^{-1}就可以通过$s_{i-1} * sv_i$得到

```cpp
s[0] = 1;
for (int i = 1; i <= n; ++i)
    s[i] = s[i - 1] * a[i] % mod;
sv[n] = qmi(s[n], p - 2);
for (int i = n; i >= 1; --i)
    sv[i - 1] = sv[i] * a[i] % mod;
for (int i = 1; i <= n; ++i)
    inv[i] = sv[i] * s[i - 1] % mod;//a[i]的逆元
```

2.5. 欧拉函数

2.5.1. 定义

$\varphi(x)$

小于等于 x,且与 x 互质的数的个数

2.5.2. 性质

1.积性函数

$\varphi(ab) = \varphi(a)\varphi(b)$

2. $n = \sum_{d|n} \varphi(d)$

3. 假如$n = p^k$,p是质数,则有$\varphi(n) = p^k - p^{k-1} = p^{k-1} * (p-1)$

证明:

在$1 \sim p^k$中,显然只有p^{k-1}个数是p的倍数,其他数都与p互质

所以就有$\varphi(n) = p^k - p^{k-1}$

4.根据算数基本定理$n = \prod_{i=1}^{s} p_i^{k_i}$, 其中$p_i$为质数,有

$$\varphi(n) = n \times \prod_{i=1}^{s} \frac{p_i - 1}{p_i}$$

证明:

因为$\varphi(n)$是积性函数,所以

$$
\begin{aligned}
n &= \prod_{i=1}^{s} \varphi\left(p_i^{k_i}\right) \\
&= \prod_{i=1}^{s} (p_i - 1) p_i^{k_i - 1} \\
&= \prod_{i=1}^{s} p_i^{k_i} \left(1 - \frac{1}{p_i}\right) \\
&= n \prod_{i=1}^{s} \frac{p_i - 1}{p_i}
\end{aligned}
$$

5. 对于任意不全为 0 的$m, n, \varphi(mn)\varphi(\gcd(m,n)) = \varphi(m)\varphi(n)\gcd(m,n)$

不懂,先抄上

2.5.3. 实现

- 1.求单个数的欧拉函数

根据性质 4

等同于分解质因数,可以 Pollard Rho 优化

时间复杂度:$O(\sqrt{n})$

```cpp
int euler_phi(int n) {
    int ans = n;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) {
            ans = ans / i * (i - 1);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        ans = ans / n * (n - 1);
    return ans;
}
```

- 2.线性筛求欧拉函数

p_1是 n 的最小质因子,$n' = \frac{n}{p_1}$

针对两种情况进行讨论:

1.$n' \bmod p_1 = 0$

则n'包含了n的所有质因子

$$\begin{aligned}\varphi(n) &= n \times \prod_{i=1}^{s} \frac{p_i - 1}{p_i} \\ &= p_1 \times n' \times \prod_{i=1}^{s} \frac{p_i - 1}{p_i} \\ &= p_1 \times \varphi(n')\end{aligned}$$

n'与p_1互质,根据性质 1 有:

$$\begin{aligned}\varphi(n) &= \varphi(p_1) \times \varphi(n') \\ &= (p_1 - 1) \times \varphi(n')\end{aligned}$$

2.$n' \bmod p_1 \neq 0$

```cpp
void pre(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i]) {
            pri.push_back(i);
            phi[i] = i - 1;
        }
        for (int pri_j : pri) {
            if (i * pri_j > n)
                break;
            not_prime[i * pri_j] = true;
            if (i % pri_j == 0) {
                phi[i * pri_j] = phi[i] * pri_j;
                break;
            }
            phi[i * pri_j] = phi[i] * phi[pri_j];
        }
    }
}
```

2.6. 费马小定理 && 欧拉定理

2.6.1. 费马小定理

若p为素数,$\gcd(a, p) = 1$, 则$a^{p-1} \equiv 1 \pmod p$

2.6.2. 证明

构造一个序列$A = \{1, 2, ..., p-1\}$,这个序列有一下性质

$$\prod_{i=1}^{p-1} A_i \equiv \prod_{i=1}^{p-1} (A_i \times a) \pmod{p}$$

证明:

$\because \gcd(A_i, a) \ = 1, \gcd(A_i \times a, p) = 1 (\text{mod } \ p)$

$\because$每个$A_i \times a \ (\text{mod } \ p)$都是独一的,并且$A_i \times a < p$

$\therefore$每一个$A_i \times a$与一个A_i对应

设$f = (p-1)!$,则$f \equiv a \times A_1 \times a \times A_2 ... (\text{mod } \ p)$

$a^{p-1} \times f \equiv f$

$a^{p-1} \qquad \equiv 1$

2.6.3. 欧拉定理

若$\gcd(a, m) = 1$, 则$a^{\varphi(m)} \equiv 1 \ (\text{mod}\, m)$

证明类似费马小定理的证明,构造一个长度为$\varphi(m)$的序列A

费马小定理是欧拉定理在 p 为质数是的弱化版

2.6.4. 拓展欧拉定理(欧拉降幂)

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)} & \gcd(a, m) = 1 \\ a^b & \gcd(a, m) \neq 1, b < \varphi(m), (\text{mod } \ m) \\ a^{b \bmod \varphi(m) + \varphi(m)} & \gcd(a, m) \neq 1, b \geq \varphi(m) \end{cases}$$

```cpp
int euler_phi(int n) {
    int ans = n;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) {
            ans = ans / i * (i - 1);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        ans = ans / n * (n - 1);
    return ans;
}
int qmi(int a, ll b, int p) {
    int res = 1;
    for (; b; b /= 2, a = 1ll * a * a % p)
        if (b & 1)
            res = 1ll * res * a % p;
    return res;
}
```

```cpp
void solve() {
    ll a, p;
    string b;
    cin >> a >> p >> b;
    int phi = euler_phi(p);
    int res = 0, f = 0;
    for (int i = 0; i < b.size(); i++) {
        int u = b[i] - '0';
        // 如果 p 在 10^9,可能会爆,要把 res 开 ll!这里为了效率没开!
        res = 1ll * res * 10 + u;
        if (res >= phi)
            f = 1, res %= phi;
    }
    if (gcd(a, p) == 1)
        cout << qmi(a, res, p) << endl;
    else {
        if (f)
            res += phi;
        cout << qmi(a, res, p) << endl;
    }
}
```

2.7. 线性同余方程

解线性同余方程$ax \equiv b \pmod{p}$

转换为不定方程$ax + p \equiv b \pmod{p}$

用 exgcd 求解即可

```cpp
ll lieq(ll a, ll b, ll p) // ax=b mod p
{
    ll x = 0, y = 0;
    ll d = exgcd(a, p, x, y);// ax+p=b mod p
    if (b % d != 0)
        return -1;
    ll k = p / d;
    x *= b / d;
    x = (x % k + k) % k;
    return x;
}
```

2.8. 线性同余方程组

求方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

的解

先考虑求解第一二个方程的解,这样就转化为了求$k-1$个方程的解

$$x = a_1 + y_1 m_1 = a_2 + y_2 m_2$$

$$-y_1 m_1 + y_2 m_2 = a_1 - a_2$$

解这个不定方程可以得到

$$-y_1 = y_0 + k\frac{m_2}{m_1, m_2}$$

$$x = -y_1 m_1 + a_1 = m_1 y_0 + k\frac{m_1 m_2}{m_1, m_2}$$

$$x \equiv m_1 y_0 \pmod{[m_1, m_2]}$$

$$x \equiv x_0 \pmod{[m_1, m_2]}$$

最后就可以得到一个通解$x \equiv x_0 \pmod{[m_1, m_2, ..., m_k]}$

2.9. 中国剩余定理(crt)

2.9.1. 定理

设$m_1, m_2, ..., m_k$是两两互质的k个正整数,

则方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

的解为

$$x \equiv \sum_{i=1}^{k} M_i' M_i a_i \pmod{M}$$

其中$M = m_1 m_2 ... m_k, M_i = \frac{M}{m_i}, M_i{}' M_i \equiv 1 \pmod{m_i}$

2.9.2. 过程

一个合法解一定是形如

$$x \equiv u_1 a_1 + u_2 a_2 + ... + u_k a_k$$

先考虑如何满足第一个$x \equiv a_1 \pmod{m_1}$

可以$a_2, a_3, ..., a_k$都乘一个m_1,这样$x \pmod{m_1}$一定是$u_1 a_1 \pmod{m_1}$

其他的方程类似

这样就得到了$u_i = M_i$

然后就只需要再乘一个$M_{i'}$使得$M_i{}' M_i \equiv 1 \pmod{m_i}$即可

https://www.luogu.com.cn/problem/P1495

```cpp
void solve() {
    ll n, mul = 1;
    __int128_t ans = 0;
    cin >> n;
    vector<int> a(n + 1), b(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i] >> b[i];
        mul *= a[i];
    }
    for (int i = 1; i <= n; i++) {
        __int128_t m = mul / a[i];
        ll x = 0, y = 0;
        exgcd(m, a[i], x, y);
        ans += b[i] * m * (x < 0 ? x + a[i] : x);
    }
    ans %= mul;
    cout << (ll)ans << endl;
}
```

2.10. 卢卡斯定理

对于一个质数p有

$$\binom{n}{m} \bmod p = \binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{m}{p} \rfloor} * \binom{n \bmod p}{m \bmod p} \bmod p$$

当组合数需要取模,n, m很大,无法递推,而p比较小时适合使用

需要预处理p范围内的组合数 时间复杂度:$O(\log n)$

```cpp
ll Lucas(ll n, ll m, ll p) {
    if (m == 0)
        return 1;
    return C[n % p][m % p] * Lucas(n / p, m / p, p) % p;
}
```

2.11. 分解质因数

2.11.1. Pollard Rho 算法

时间复杂度:

$O\left(n^{\frac{1}{4}} \log n\right)$

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 10;
const ll mod = 998244353;
mt19937_64 rnd(chrono::duration_cast<chrono::nanoseconds>
(chrono::system_clock::now().time_since_epoch()).count());
ll qmi(__int128_t a, ll b, ll p) {
    ll res = 1;
    while (b) {
        if (b & 1)
            res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}
bool MR(ll n) {
    if (n == 2)
        return true;
    if (n <= 1 || n % 2 == 0)
        return false;
    const ll base[7] = {2, 325, 9375, 28178, 450775, 9780504,
1795265022};
    ll u = n - 1, k = 0;
    while (u % 2 == 0)
```

```
        k++, u /= 2;
    for (auto &x : base) {
        if (x % n == 0)
            continue;
        ll v = qmi(x, u, n);
        if (v == 1 || v == n - 1)
            continue;
        for (int j = 1; j <= k; j++) {
            ll lst = v;
            v = (__int128_t)v * v % n;
            if (v == 1) {
                if (lst != n - 1)
                    return false;
                break;
            }
        }
        if (v != 1)
            return false;
    }
    return true;
}
ll PR(ll n) {
    uniform_int_distribution<ll> num(1, n - 1);
    ll c = num(rnd);
    auto f = [&](ll x) { return ((__int128_t)x * x + c) % n; };
    ll x = 0, y = 0, s = 1;
    for (int k = 1;; k <<= 1, y = x, s = 1) {
        for (int i = 1; i <= k; i++) {
            x = f(x);
            s = (__int128_t)s * abs(x - y) % n;
            if (i % 127 == 0) {
                ll d = gcd(s, n);
                if (d > 1)
                    return d;
            }
        }
        ll d = gcd(s, n);
        if (d > 1)
            return d;
```

```
    }
    return n;
}

void solve() {
    ll n;
    cin >> n;
    vector<ll> fac;
    auto getpr = [&](auto &getpr, ll n) -> void {
        if (n == 1)
            return;
        if (MR(n)) {
            fac.push_back(n);
            return;
        }
        ll x = n;
        while (x == n)
            x = PR(n);
        getpr(getpr, x), getpr(getpr, n / x);
    };
    getpr(getpr, n);
    if (fac.size() == 1)
        cout << "Prime" << endl;
    else
        cout << *max_element(fac.begin(), fac.end()) << endl;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    cin >> _;
    while (_--)
        solve();
    return 0;
}
```

2.12. 筛法

2.12.1. 线性筛

每个非质数都只会被最小的质因子筛掉

时间复杂度:$O(n)$

```cpp
vector<int> pr;
vector<bool> not_pr(N);
auto getpr = [&](int n)
{
    for (int i = 2; i <= n; ++i)
    {
        if (!not_pr[i])
            pr.push_back(i);
        for (int p : pr)
        {
            if (i * p > n)
                break;
            not_pr[i * p] = true;
            if (i % p == 0) // 说明 i*p 已经被一个更小的 i 判断过了
                break;
        }
    }
};
```

3. 组合数学

3.1. 排列组合

3.1.1. 求组合数

- 求 1~n 的任意组合数

时间复杂度:$O(n)$

```cpp
struct Binom {
    int mx;
    vector<ll> fac, invfac;
    ll qmi(ll a, ll b) {
        ll res = 1;
        while (b) {
```

```
            if (b & 1)
                res = res * a % mod;
            b >>= 1;
            a = a * a % mod;
        }
        return res;
    }
    Binom(int n) : mx(n), fac(n + 10), invfac(n + 10) {
        fac[0] = 1;
        for (int i = 1; i <= n; i++)
            fac[i] = fac[i - 1] * i % mod;
        invfac[n] = qmi(fac[n], mod - 2);
        for (int i = n - 1; i >= 0; i--)
            invfac[i] = invfac[i + 1] * (i + 1) % mod;
    }
    ll C(int n, int m) {
        if (n < m || m < 0 || n > mx)
            return 0ll;
        return fac[n] * invfac[n - m] % mod * invfac[m] % mod;
    }
    ll A(int n, int m) {
        if (n < m || m < 0 || n > mx)
            return 0ll;
        return fac[n] * invfac[n - m] % mod;
    }
};
```

- 直接求组合数

```cpp
auto C = [&](ll a, ll b) -> ll
{
    if (a < b)
        return 0ll;
    __int128_t res = 1;
    for (ll i = a, j = 1; j <= b; i--, j++)
        res = res * i / j;
    return (ll)res;
};
```

- 递推

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

```cpp
for (int i = 0; i <= n; i++)
    for (int j = 0; j <= i; j++)
        if (!j)
            c[i][j] = 1;
        else
            c[i][j] = (c[i - 1][j] + c[i - 1][j - 1]) % mod;
```

3.1.2. 插板法

- n 个 相同 元素分成非空的 k 组

n-1 个空隙插入 k-1 个板子

$$\binom{n-1}{k-1}$$

- n 个 相同 元素分成 k 组(可为空)

先加入 k 个元素,相当于保证每组至少一个,这样就转换为上一个问题,分组后再抽掉

$$\binom{n+k-1}{k-1} = \binom{n+k-1}{n}$$

3.1.3. 组合数性质/二项式推论

1.

$$\binom{n}{m} = \binom{n}{n-m}$$

所选集合对全集取补集(对称性)

n 取 m = n 剩 n-m = n 取 n-m

2.

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

$\because \binom{n}{m} = \frac{n!}{m!(n-m)!}, \binom{n-1}{m-1} = \frac{(n-1)!}{(m-1)!(n-m)!}$

$\therefore \binom{n}{m} = \frac{n}{m}\binom{n-1}{m-1}$

3.

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

由定义得到的递推式

要么不取最新的一个,就是$\binom{n-1}{m}$

要么就是一定取最新的,就是$\binom{n-1}{m-1}$

3.2. 容斥原理

容斥系数:-1^k

3.3. 斯特林数

第二类斯特林数

n 个不同的元素,分成 m 个集合的方案数

- 递推式

$s(n,m)=s(n-1,m)+(m-1)*s(n-1,m)$

多出来的一个元素，要么放进一个新的集合，要么放进原有的一个集合当中

- 通项公式

先考虑一个弱化的情况，n 个不同元素，放入 m 个不同集合，集合可为空

m^n

从集合不同到集合相同，只需要再除以 m!即可

利用容斥，再变成-1 个空盒+2 空盒-3 个空盒。。。

$s(n,m)=\frac{1}{m!}\sum_{k=0}^{m}-1^k\binom{m}{k}(m-k)^n$

3.4. 范德蒙卷积

$\sum_{i=0}^{k}\binom{n}{i}\binom{m}{k-i}=\binom{n+m}{k}$

证明:

1.组合意义

从大小为$n+m$的集合中取出k个数,可以等效于把集合拆成两个集合,大小分别为n与m,然后从n中取i个数,从m中取出$k-i$个数的方案数.

因为枚举了i,所以只需要考虑一种拆法,不同拆法之间是等效的.

2.二项式定理

不懂,待补

3.5. 卡特兰数

长度为 n 的合法括号序列的数量,特别的,$H_1=1$

定义式

$$H_n = \sum_{i=0}^{n-1} H_i H_{n-i-1}, H_0 = 1$$

相关式子

$$H_n = \binom{2n}{n} - \binom{2n}{n-1}$$

$$H_n = \frac{\binom{2n}{n}}{n+1}$$

$$H_n = \frac{4n-2}{n+1} H_{n-1}$$

3.6. 分拆数

3.6.1. 定义

将自然数k写成递降正整数和的表示

$$n = a_1 +_2 +a_3 + \ldots + a_k (a_1 \geq a_2 \geq \ldots \geq a_k \geq 1)$$

记作p_n

3.6.2. k 部分分拆数

将n分成 k 部分的分拆数,称为k部分分拆数,记作$p(n,k)$

$$n = a_1 +_2 +a_3 + \ldots + a_k (a_1 \geq a_2 \geq \ldots \geq a_k \geq 1)$$

$$n - k = a_1 +_2 +a_3 + \ldots + a_k (a_1 \geq a_2 \geq \ldots \geq a_k \geq 0)$$

如果方程恰有j部分非 0,则恰有$p(n-k,j)$个解

所以有以下式子

$$p(n,k) = \sum_{j=0}^{k} p(n-k,j$$

```cpp
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        f[i][j] = (f[i - 1][j - 1] + f[i - j][j]) % mod;
```

3.6.3. 互异分拆数

自然数n分成k部分,且各部分两两不同的方案数,记作pd_n

$$n = a_1 +_2 +a_3 + \ldots + a_k (a_1 > a_2 > \ldots > a_k \geq 1)$$

$$n-k=a_1+_2+a_3+...+a_k(a_1>a_2>...>a_k\geq 0)$$

至多一项为 0,所有有以下式子

$$pd(n,k)=pd(n-k,k-1)+pd(n-k,k)$$

4. 博弈论

通过性质将大问题化简为小问题(n->n-2)

Who is HB?

4.1. 基础知识

平等/不平等博弈:

不同人对状态的影响是否一样

必胜态:

存在后继必败

必败态:

所有后继必胜

公平组合游戏(Impartial Combinatorial Games,ICG)

1. 有两名玩家
2. 两名玩家轮流操作，在一个有限集合内任选一个进行操作，改变游戏当前局面
3. 一个局面的合法操作，只取决于游戏局面本身且固定存在，与玩家次序或者任何其它因素无关
4. 无法操作者，即操作集合为空，输掉游戏，另一方获胜

4.2. 公平组合游戏

4.2.1. Nim 游戏

n 堆物品，每堆有 a_i 个，两个玩家轮流取走任意一堆的任意个物品，但不能不取。 取走最后一个物品的人获胜。

4.2.2. SG 函数

SG(S)=mex(S)=min{x}(x ∉ S,x∈ N)

```cpp
int mex(auto v) // v 可以是 vector、set 等容器
{
    unordered_set<int> S;
    for (auto e : v)
        S.insert(e);
```

```
    for (int i = 0;; ++i)
        if (S.find(i) == S.end())
            return i;
}
```

- sg 定理

对于由 n 个有向无环图组成的组合游戏,设它们的起点为s_i,有:

当且仅当 $sg(s_1) \oplus sg(s_2) \oplus ... \oplus sg(s_n)$!=0 时,这个游戏是先手必胜的。

5. 线性代数

5.1. 矩阵

- 线性运算+/-/数乘
- 乘法

封装模板

```cpp
struct Mat
{
    int sz;
    vector<vector<ll>> a;
    Mat(int _sz) : sz(_sz), a(_sz + 10, vector<ll>(_sz + 10, 0)) {}
    Mat operator-(const Mat &T) const
    {
        Mat res(sz);
        for (int i = 1; i <= sz; i++)
            for (int j = 1; j <= sz; j++)
                res.a[i][j] = (a[i][j] - T.a[i][j]) % mod;
        return res;
    }
    Mat operator+(const Mat &T) const
    {
        Mat res(sz);
        for (int i = 1; i <= sz; i++)
            for (int j = 1; j <= sz; j++)
                res.a[i][j] = (a[i][j] + T.a[i][j]) % mod;
        return res;
    }
    Mat operator*(const Mat &T) const
```

```
    {
        Mat res(sz);
        for (int i = 1; i <= sz; i++)
            for (int k = 1; k <= sz; k++)
                if (a[i][k])
                    for (int j = 1; j <= sz; j++)
                        if (T.a[k][j])
                            res.a[i][j] += (a[i][k] * T.a[k][j]) % mod, res.a[i][j] %= mod;
        return res;
    }
    Mat operator^(ll x) const
    {
        Mat res(sz), bas(sz);
        for (int i = 1; i <= sz; ++i)
            res.a[i][i] = 1;
        for (int i = 1; i <= sz; ++i)
            for (int j = 1; j <= sz; ++j)
                bas.a[i][j] = a[i][j] % mod;
        while (x)
        {
            if (x & 1)
                res = res * bas;
            bas = bas * bas;
            x >>= 1;
        }
        return res;
    }
};
```

6. 高等数学

6.1. 调和级数

$$\sum_{i=1}^{x} \frac{1}{i} = O(\ln x)$$

$$\sum_{i=1}^{n} \frac{n}{i} = O(n \ln x)$$

7. 多项式与生成函数

7.1. 积性函数

若$f(n), g(n)$是积性函数,则$h(n) = f(n)g(n)$也是积性函数

求积性函数

用质因数分解求$f(n)$

```cpp
ll get_f(ll n) {
    ll ans = 1;
    for (int i = 2; i <= n / i; i++) {
        int cnt = 0;
        while (n % i == 0)
            cnt++, n /= i;
        ans *= f(i, cnt);
    }
    if (n > 1)
        ans *= f(n, 1);
    return ans;
}
```

用欧拉筛求$f(1), ..., f(n)$

---

8. 结论

1. 有 s 种不同类型的糖果,总数为 n,两个不同的糖果可以配对成一组,问最多配对多少组.

如果数量最多的糖果不超过总数的一半,那我们总是可以配对出$\lfloor \frac{n}{2} \rceil$组.

否则我们就无法用完最大的糖果,假设最大的糖果有 mx 个,那我们只能配对出 n-mx 组.

求方案:按照数量排序，先放奇数位，再放偶数位

2.一个数组和为 s，最多有$\sqrt{s}$种不同的数

3.$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$